

EXPERIMENT NO. 7 - MongoDB

Name of Student	Sonam Chhabaidiya
Class Roll No	D15A_09
D.O.P.	
D.O.S.	
Sign and Grade	

AIM: To study CRUD operations in MongoDB

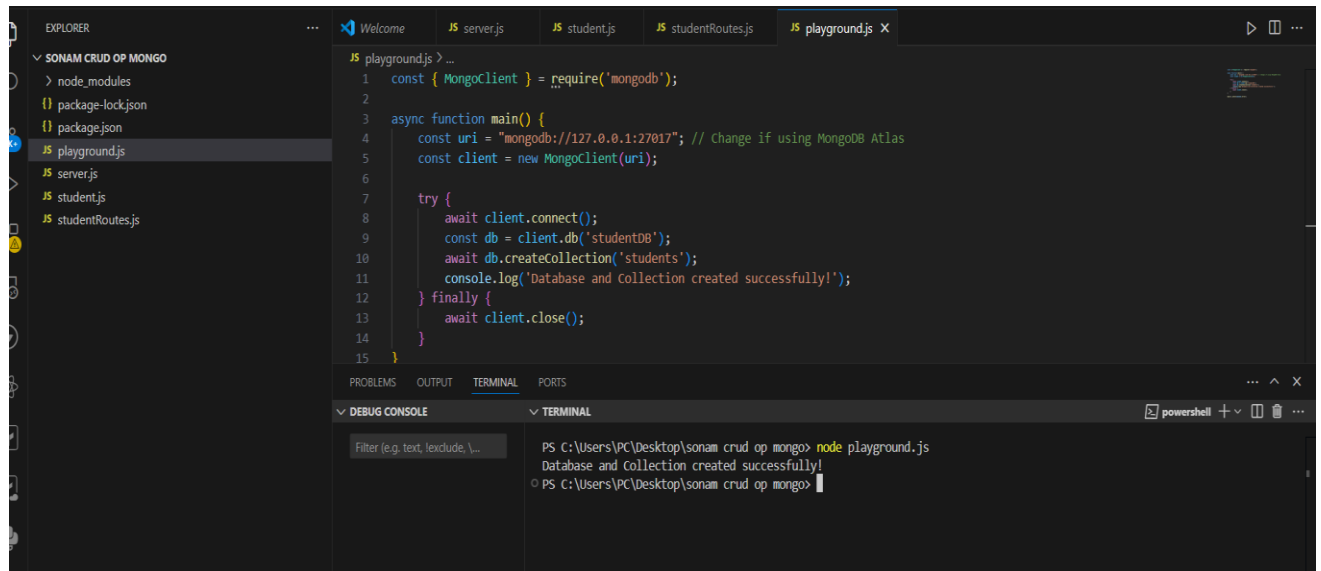
PROBLEM STATEMENT:

1. Create a new database to storage student details of IT dept(Name, Roll no, class name) and perform the following on the database
 - a. Insert one student details
 - b. Insert at once multiple student details
 - c. Display student for a particular class
 - d. Display students of specific roll no in a class
 - e. Change the roll no of a student
 - f. Delete entries of particular student
2. Create a set of RESTful endpoints using Node.js, Express, and Mongoose for handling student data operations. The endpoints should support:
 - a. Retrieve a list of all students.
 - b. Retrieve details of an individual student by ID.
 - c. Add a new student to the database.
 - d. Update details of an existing student by ID.
 - e. Delete a student from the database by ID.Connect the server to MongoDB using Mongoose, and store student data with attributes: name, age, and grade.

GITHUB LINK: <https://github.com/sonamcc/webx7>

OUTPUT:

1. Create a database to store student details of IT Department

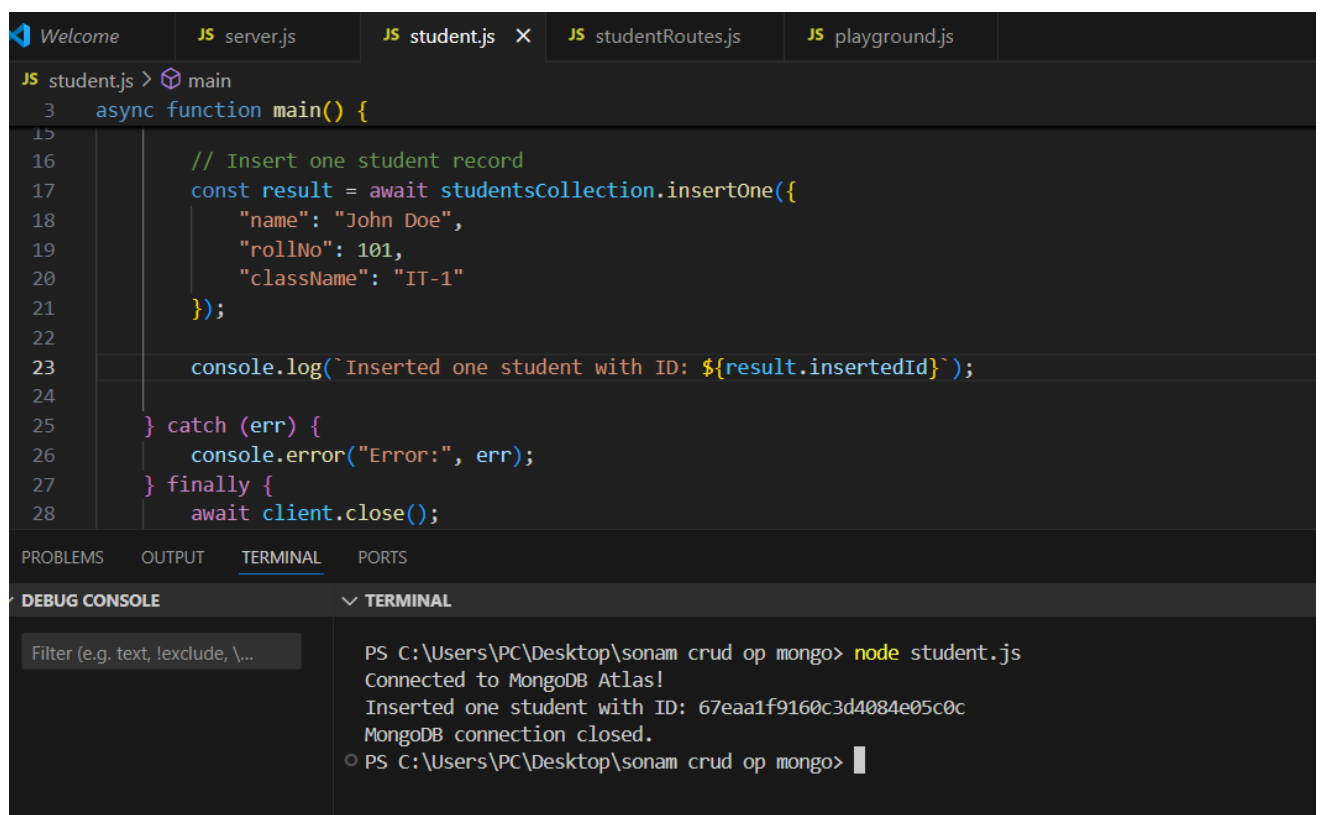


The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'SONAM CRUD OP MONGO' with files: 'node_modules', 'package-lock.json', 'package.json', 'playground.js', 'server.js', 'student.js', and 'studentRoutes.js'. The code editor shows the 'playground.js' file with the following code:

```
1  const { MongoClient } = require('mongodb');
2
3  async function main() {
4    const uri = "mongodb://127.0.0.1:27017"; // Change if using MongoDB Atlas
5    const client = new MongoClient(uri);
6
7    try {
8      await client.connect();
9      const db = client.db('studentDB');
10     await db.createCollection('students');
11     console.log('Database and collection created successfully!');
12   } finally {
13     await client.close();
14   }
15 }
```

The terminal at the bottom shows the command 'node playground.js' and the output: 'Database and Collection created successfully!'.

a) Insert one Student Detail

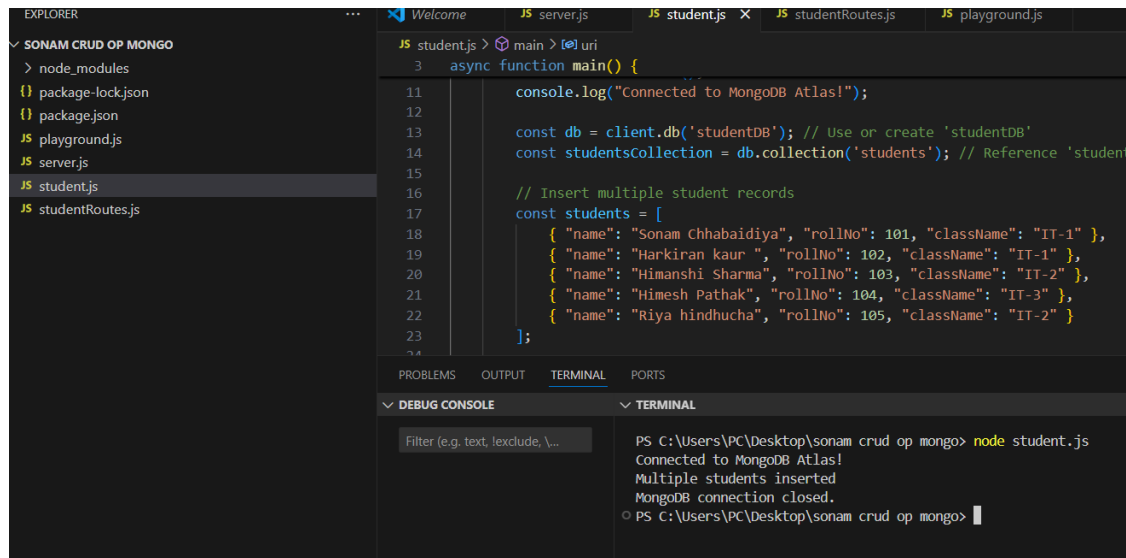


The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'SONAM CRUD OP MONGO' with files: 'node_modules', 'package-lock.json', 'package.json', 'playground.js', 'server.js', 'student.js', and 'studentRoutes.js'. The code editor shows the 'student.js' file with the following code:

```
1  // Insert one student record
2  const result = await studentsCollection.insertOne({
3    "name": "John Doe",
4    "rollNo": 101,
5    "className": "IT-1"
6  });
7
8  console.log(`Inserted one student with ID: ${result.insertedId}`);
9
10 } catch (err) {
11   console.error("Error:", err);
12 } finally {
13   await client.close();
14 }
```

The terminal at the bottom shows the command 'node student.js' and the output: 'Connected to MongoDB Atlas!', 'Inserted one student with ID: 67eaa1f9160c3d4084e05c0c', and 'MongoDB connection closed.'.

b) Insert at once multiple Student Details



The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'SONAM CRUD OP MONGO' with files like 'package-lock.json', 'package.json', 'playground.js', 'server.js', 'student.js', and 'studentRoutes.js'. The code editor shows the 'student.js' file with the following code:

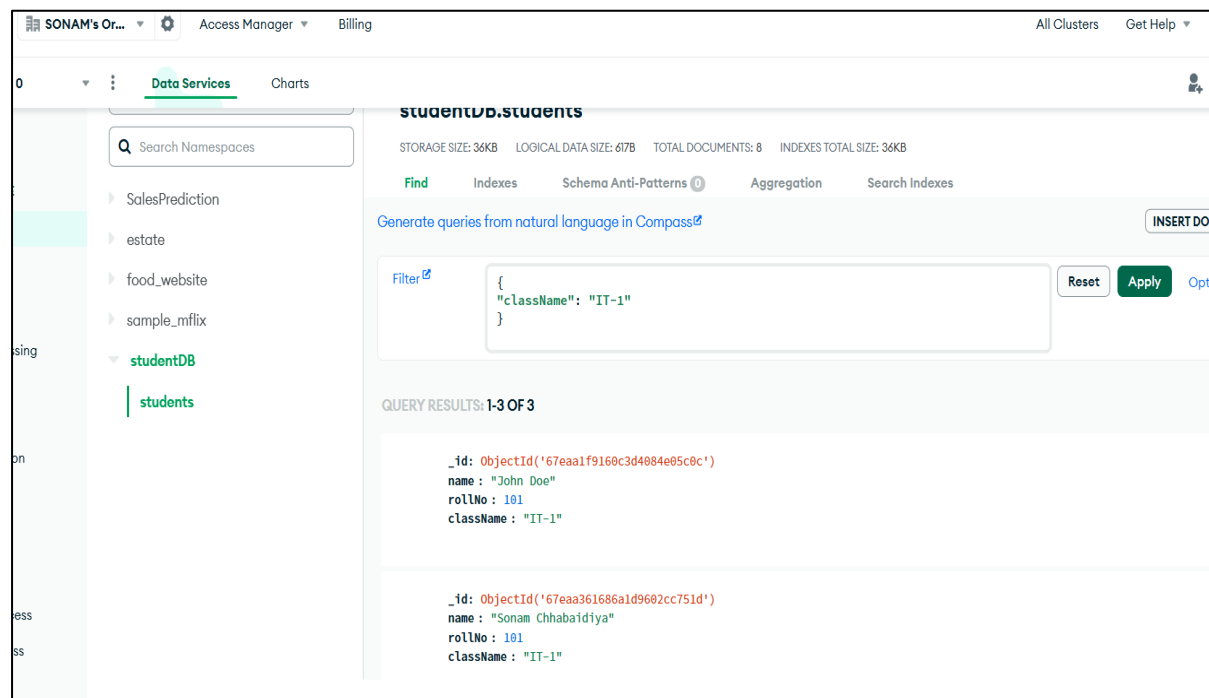
```
3  async function main() {
11      console.log("Connected to MongoDB Atlas!");
12
13      const db = client.db('studentDB'); // Use or create 'studentDB'
14      const studentsCollection = db.collection('students'); // Reference 'student
15
16      // Insert multiple student records
17      const students = [
18          { "name": "Sonam Chhabaidiya", "rollNo": 101, "className": "IT-1" },
19          { "name": "Harkiran kaur ", "rollNo": 102, "className": "IT-1" },
20          { "name": "Himanshi Sharma", "rollNo": 103, "className": "IT-2" },
21          { "name": "Himesh Pathak", "rollNo": 104, "className": "IT-3" },
22          { "name": "Riya hindhucha", "rollNo": 105, "className": "IT-2" }
23      ];
24  }
```

Below the code editor, there is a 'TERMINAL' tab showing the output of the command 'node student.js'. The output is:

```
PS C:\Users\PC\Desktop\sonam crud op mongo> node student.js
Connected to MongoDB Atlas!
Multiple students inserted
MongoDB connection closed.
PS C:\Users\PC\Desktop\sonam crud op mongo>
```

c) Display Students for a Particular Class

```
{
  "className": "IT-1"
}
```



d) Display Student of a Specific Roll Number in a Class

```
{  
  "className": "IT-1",  
  "rollNo": 101  
}
```

The screenshot shows the MongoDB Atlas web interface. On the left, the 'Data Services' sidebar is open, showing a tree view of namespaces including 'studentDB' and 'students'. The main panel displays the 'studentDB.students' collection. A filter is applied: `{ "className": "IT-1", "rollNo": 101 }`. Below the filter, two documents are shown:

```
{  
  "_id": ObjectId('67eaa1f9160c3d4084e05c0c'),  
  "name": "John Doe",  
  "rollNo": 101,  
  "className": "IT-1"  
}
```

```
{  
  "_id": ObjectId('67eaa361686a1d9602cc751d'),  
  "name": "Sonam Chhabaidiya",  
  "rollNo": 101,  
  "className": "IT-1"  
}
```

e) Change the Roll Number of a Student

Filter:

```
{  
  "name": "John Doe"  
}
```

Update:

The screenshot shows the MongoDB Atlas web interface. The filter is updated to: `{ "name": "John Doe" }`. The 'QUERY RESULTS: 1-1 OF 1' section shows the updated document:

```
1  _id: ObjectId('67eaa1f9160c3d4084e05c0c')  
2  name: "John Doe"  
3  rollNo: 103  
4  className: "IT-1"
```

A yellow banner at the bottom indicates: "Document modified."

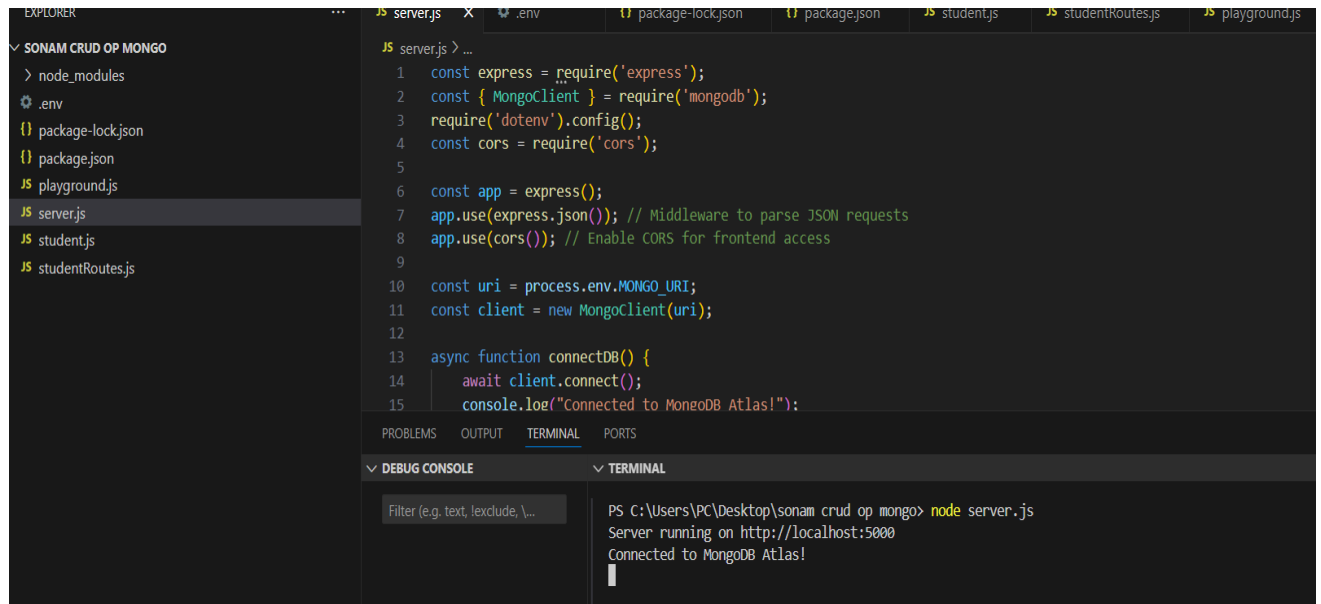
f) **Delete Entries of a Particular Student**
Filter:

```
{  
  "name": "Himesh Pathak "  
}
```

Delete:

The screenshot displays the SONAM Data Services interface. On the left, a sidebar shows a tree view of namespaces, with 'studentDB' and its sub-namespace 'students' selected. The main panel is titled 'studentDB.students' and shows database statistics: STORAGE SIZE: 36KB, LOGICAL DATA SIZE: 67B, TOTAL DOCUMENTS: 8, and INDEXES TOTAL SIZE: 36KB. Below these are tabs for 'Find', 'Indexes', 'Schema Anti-Patterns', 'Aggregation', and 'Search Indexes'. A 'Generate queries from natural language in Compass' link is present. A 'Filter' input field contains the JSON query: `{ "name": "Himesh Pathak" }`. To the right of the filter are 'Reset', 'Apply', and 'Options' buttons. Below the filter, the 'QUERY RESULTS: 1-1 OF 1' section shows a single document with the following fields: `_id: ObjectId('67eaa361686a1d9602cc7520')`, `name: "Himesh Pathak"`, `rollNo: 104`, and `className: "IT-3"`. At the bottom, a red banner indicates 'Document flagged for deletion.' with 'CANCEL' and 'DELETE' buttons.

2. Restful api



The screenshot shows a VS Code editor with a project named "SONAM CRUD OP MONGO". The file explorer on the left lists files: node_modules, .env, package-lock.json, package.json, playground.js, server.js, student.js, and studentRoutes.js. The main editor displays the content of server.js, which is a Node.js Express application. The code includes imports for express, MongoClient, dotenv, and cors. It sets up an Express app, uses JSON middleware and CORS, and connects to a MongoDB Atlas instance. The terminal at the bottom shows the command "node server.js" being executed, with output indicating the server is running on http://localhost:5000 and connected to MongoDB Atlas.

```
server.js > ...
1  const express = require('express');
2  const { MongoClient } = require('mongodb');
3  require('dotenv').config();
4  const cors = require('cors');
5
6  const app = express();
7  app.use(express.json()); // Middleware to parse JSON requests
8  app.use(cors()); // Enable CORS for frontend access
9
10 const uri = process.env.MONGO_URI;
11 const client = new MongoClient(uri);
12
13 async function connectDB() {
14   await client.connect();
15   console.log("Connected to MongoDB Atlas!");
16 }
```

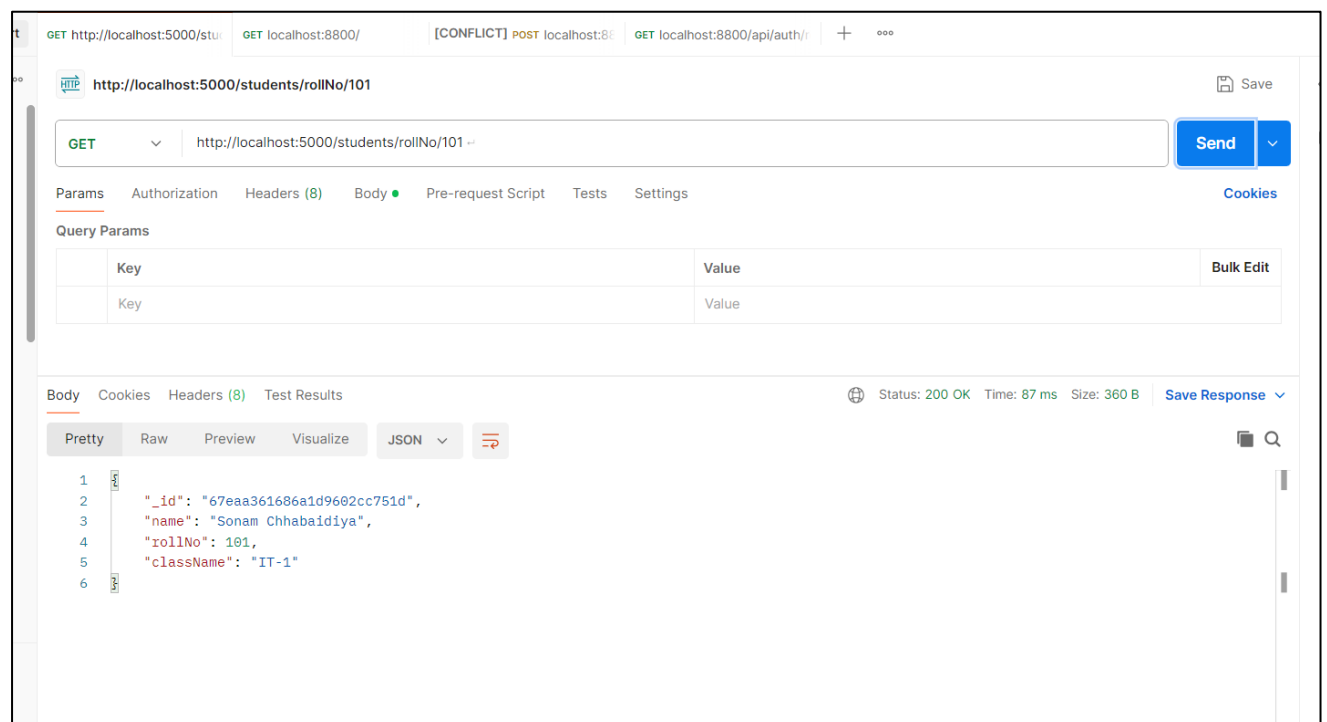
PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

Filter (e.g. text, lexclude, \...

PS C:\Users\PC\Desktop\sonam crud op mongo> node server.js
Server running on http://localhost:5000
Connected to MongoDB Atlas!

A) Retrieve details of an individual student by Roll No.



The screenshot shows the Postman interface for a REST client. The URL bar shows a GET request to "http://localhost:5000/students/rollNo/101". The request is sent, and the response is displayed in the "Body" tab. The response is a JSON object containing student details. The status is 200 OK, the time is 87 ms, and the size is 360 B.

GET http://localhost:5000/students/rollNo/101

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (8) Test Results

Status: 200 OK Time: 87 ms Size: 360 B Save Response

Pretty Raw Preview Visualize JSON

```
1  {
2    "_id": "67eaa361686a1d9602cc751d",
3    "name": "Sonam Chhabaidiya",
4    "rollNo": 101,
5    "className": "IT-1"
6  }
```

b) Add a new student to the database

The screenshot shows a REST client interface with a POST request to `http://localhost:5000/students`. The request body is a JSON object representing a new student:

```
{  "id": "660f1124c5a2aeb45b12e5c3",  "name": "Baikha Chhabadiya",  "rollNo": 107,  "className": "IT-1"}
```

The response is a 201 Created status with the following JSON body:

```
{  "message": "Student added successfully",  "student": {    "name": "Baikha Chhabadiya",    "rollNo": 107,    "className": "IT-1",    "id": "67eab84ba18d21a9b88591fc"  }}
```

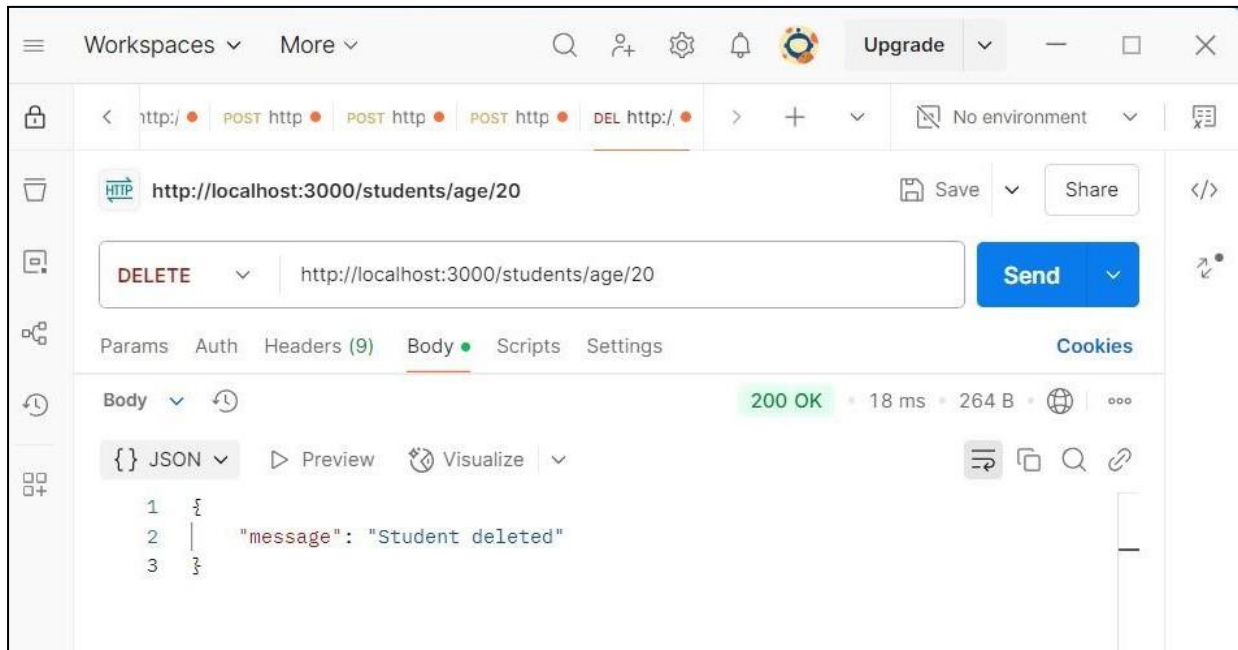
c) Update details of an existing student by ID

The screenshot shows a REST client interface with a PUT request to `http://localhost:3000/students/age/21`. The request body is a JSON object representing the updated student details:

```
{  "_id": "67e828cec4578f3b4cc5b1b1",  "name": "Anuprita Mhapankar Updated",  "age": 20,  "grade": "A+",  "__v": 0}
```

The response is a 200 OK status with a response time of 286 ms and a size of 336 B.

d) Delete a student from the database by ID.



CONCLUSION

In this experiment, we successfully performed CRUD operations in **MongoDB** and implemented a **RESTful API** using **Node.js**, **Express**, and **Mongoose**. We learned how to create, read, update, and delete student records both via **MongoDB shell commands** and **API endpoints**.